IMEN891K / IDSC501 / AXGS-501 · DATA ENGINEERING · FALL 2026

14.

Big Data II: Data Warehousing

Sangwoo Mo · POSTECH
Industrial & Management Engineering

Where We Are

PART I

Collection and Preparation
Lec 2-5

PART II

SQL
Lec 6-8

NoSQL
Lec 9-10

Big Data
Lec 13-15

PART III

Monitoring and Updates
Lec 16-19

Recap: Processing at scale

MapReduce (Lecture 13)

```
map(k, v) -> list(k2, v2)       // user
shuffle: group by k2             //
system, via disk
reduce(k2, list(v2)) -> output   // user
```

Spark (Lecture 13)

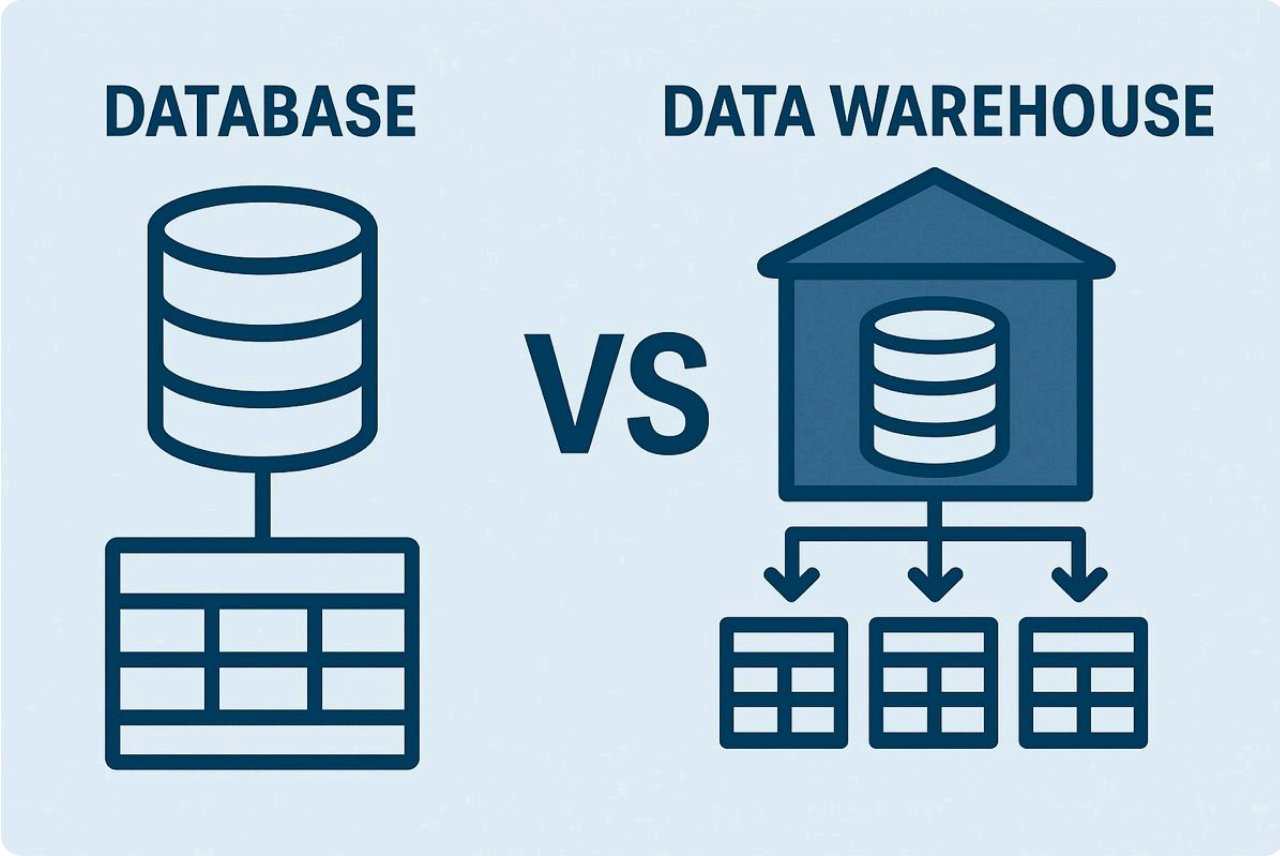
```
RB = R.filter(lambda t: t[1] > 200)
# lazy
SC = S.filter(lambda t: t[1] < 100)
J = RB.join(SC)
J.count()  # action
df.groupBy("name").avg("age")  #
lazy DataFrame
```

Both split the work across servers; both leave the storage question open: **where do the tables live, and in what layout?** That is today.

Outline: Big Data II

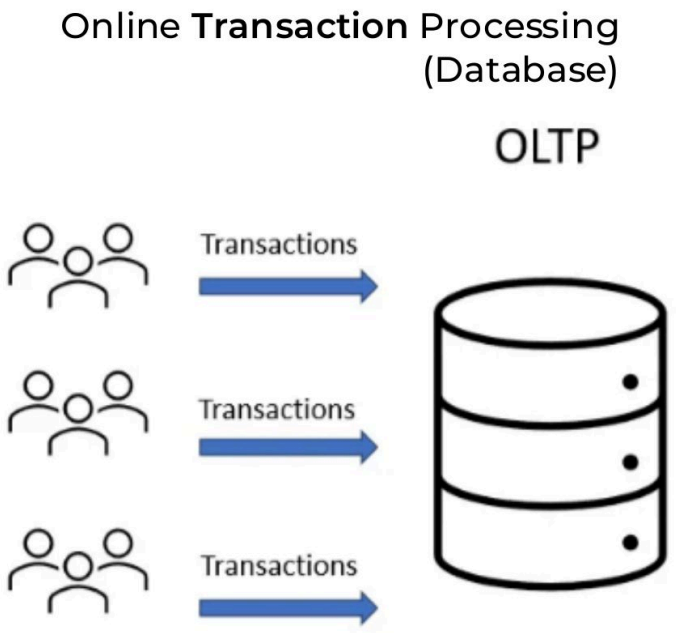
01	**Warehouses for Analytics**	organize data for analytical queries
02	**Querying a Warehouse**	query large tables with managed tools
03	**From Files to Analytical Tables**	connect raw files to usable tables
04	**Running Reliable Data Pipelines**	run, check, and recover data jobs

Warehouse Purpose



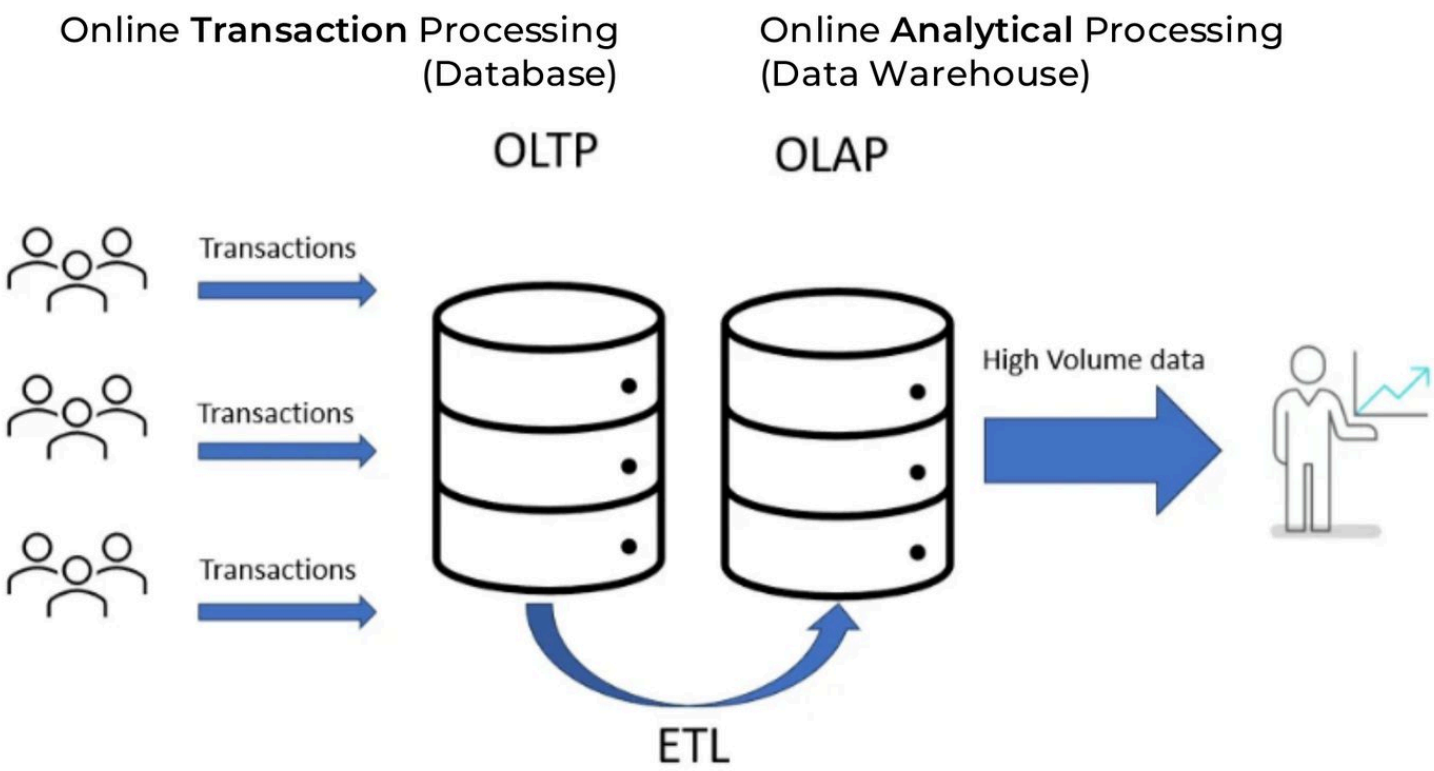
A warehouse collects from many databases and serves **analysis**, not operations

OLTP Workload



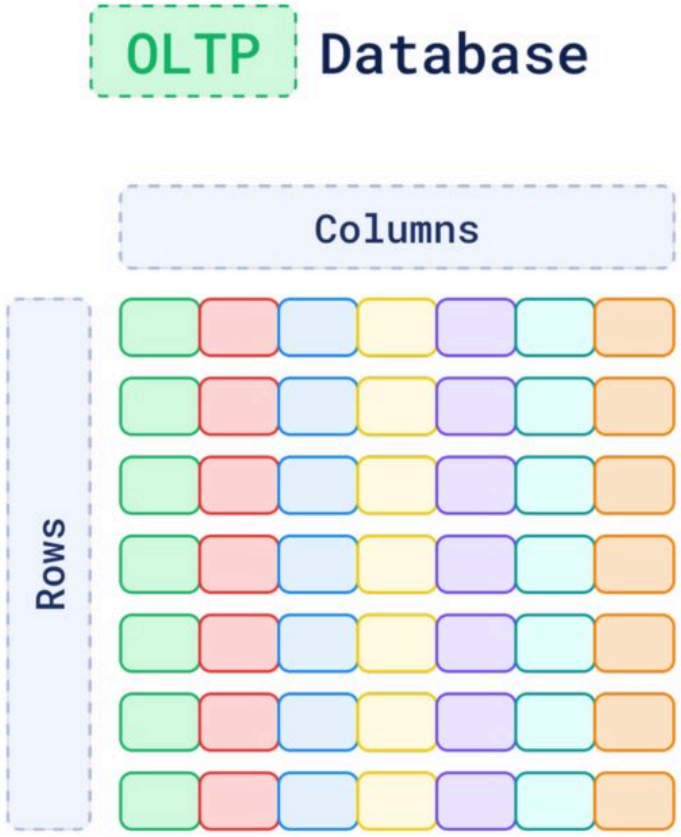
OLTP: many users, each touching a few rows, right now

OLAP Workload



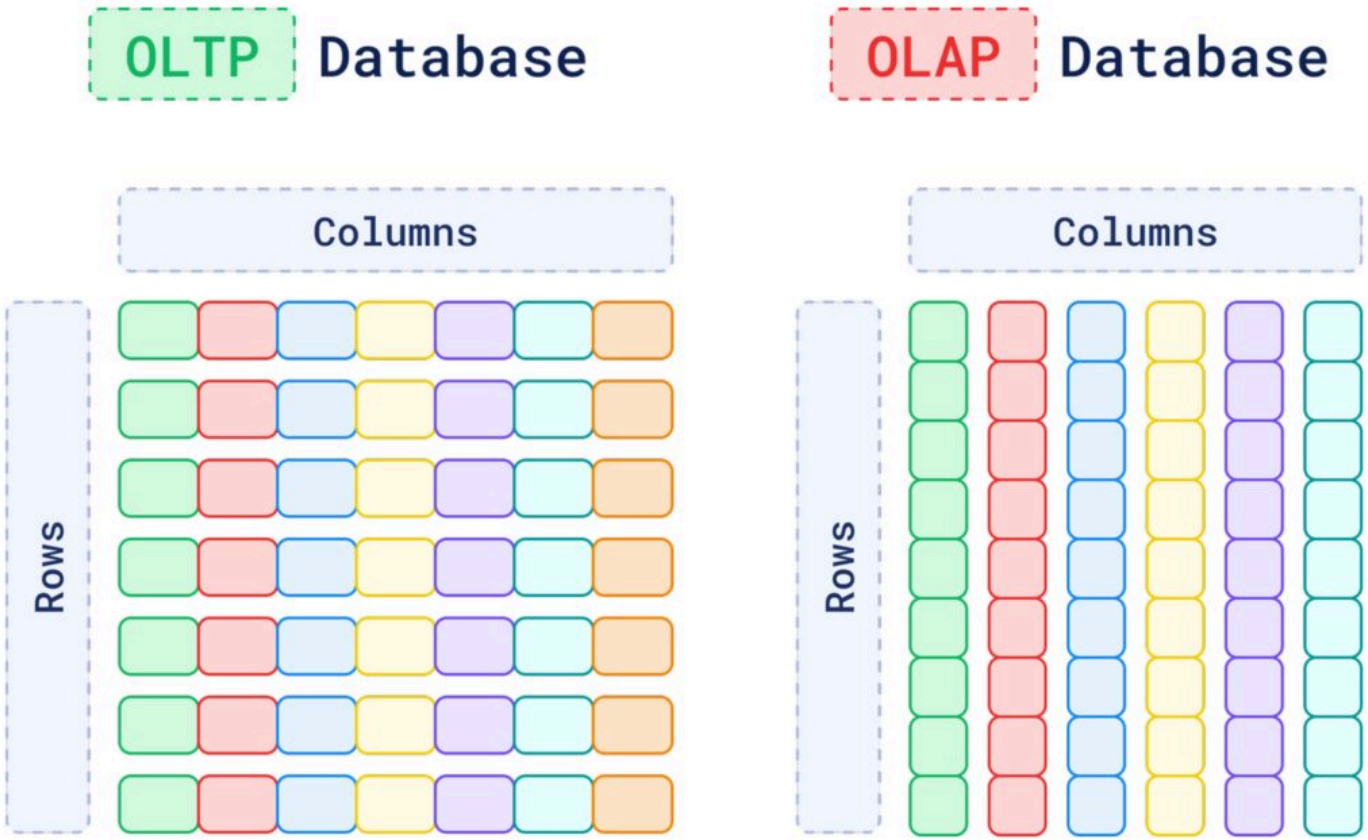
OLAP: a few analysts, each scanning millions of rows; ETL moves data across

Row Storage



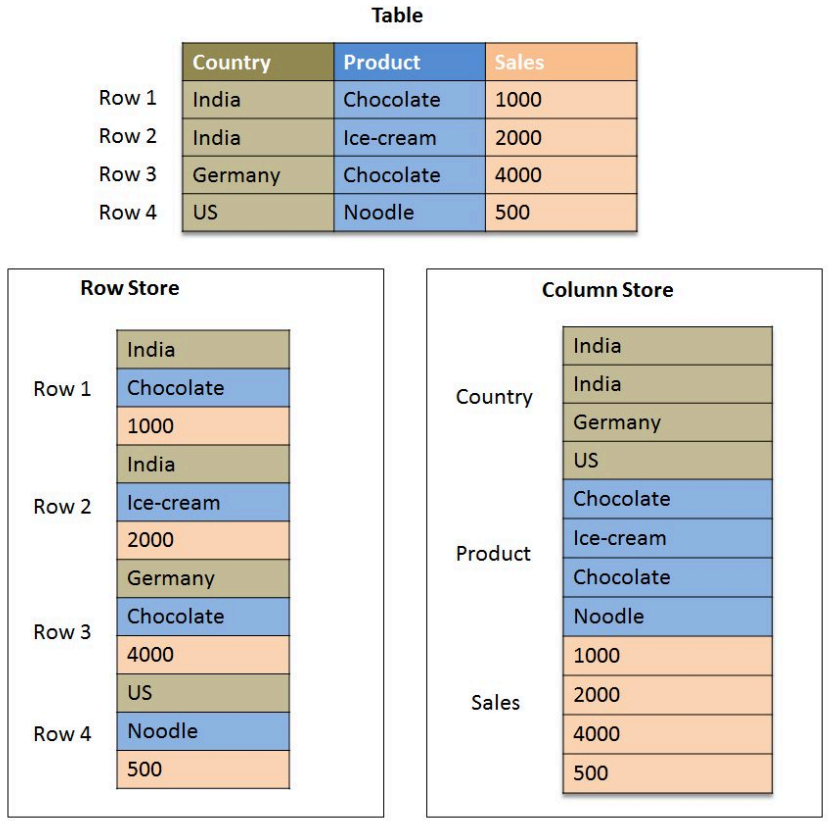
OLTP stores **whole rows together**

Column Storage



OLAP stores **whole columns together**: same table, different layout

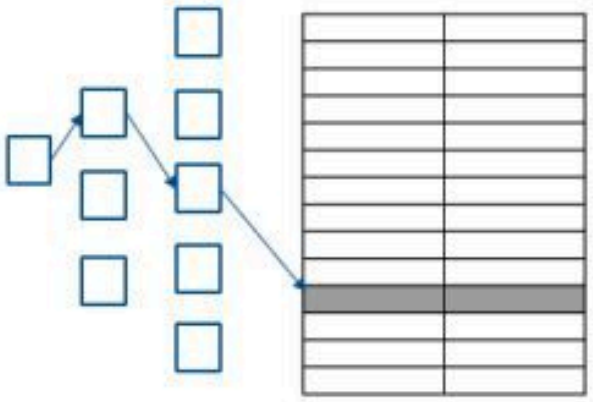
Rows and Columns on Disk



Twelve values, two orders on disk

When Each Layout Wins

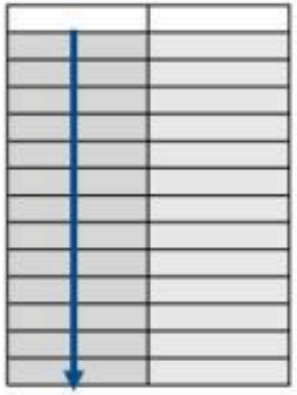
Row-oriented storage & execution



```
SELECT * FROM orders
WHERE orderid = $1
```

At scale: Fast on OLTP, Slow on OLAP

Column-oriented storage & execution



```
SELECT productid, count(*) FROM orders
GROUP BY 1 ORDER BY 2 DESC LIMIT 10
```

At scale: Slow on OLTP, Fast on OLAP

One row: row store wins. One column over all rows: **column store wins**

Index

A sorted copy of one column, pointing back at the rows

```
CREATE INDEX
idx_machine
    ON
events(machine);
```

Without an index, `WHERE machine = 'M17'` reads **every row**: a full scan.

An index on `machine` is a separate structure, sorted by that column, that lists the rows holding each value. The database searches it like a dictionary (a B-tree: log n steps) and reads only those rows.

Lecture 6's optimizer slide chose between "scan" and "use the index". This is the index it meant.

idx_machine (sorted)

machine	rowids
M01	12, 57, 91, …
M02	3, 48, …
…	…
M17	7, 30, 66, …
…	…

Sorted by the indexed column, pointing back at the rows: **a dictionary for one column**

Demo: Scan vs Index

The index that Lecture 6 assumed, measured

1. Run the count of rows for machine `M17` as given and note the time in the status line.
2. Add `CREATE INDEX idx ON events(machine);` before the query and run again. Compare the milliseconds.
3. Now try `SELECT AVG(temp) FROM events` with and without the index. Does the index help? Why not?

```
SELECT COUNT(*) FROM events WHERE machine =
'M17';
```

QUERY RESULT

COUNT(*)
465

Answer: Scan vs Index

```
SELECT COUNT(*)
FROM events WHERE
machine = 'M17';
```

COUNT(*)
465

```
CREATE INDEX idx ON
events(machine);
EXPLAIN QUERY PLAN
SELECT COUNT(*)
FROM events WHERE
machine = 'M17';
```

detail
SEARCH events USING COVERING INDEX idx (machine=?)

The plan changes from **SCAN events** to **SEARCH events USING COVERING INDEX idx**. An aggregate over every row (`AVG(temp)`) still scans: an index speeds up *finding* rows, not *reading all of them*.

SCAN becomes SEARCH … USING INDEX; an aggregate over all rows **still scans**

Column Compression

Sorted columns repeat, and repeats compress

A column holds **one type**; sorted, equal values sit together. Two encodings use that:

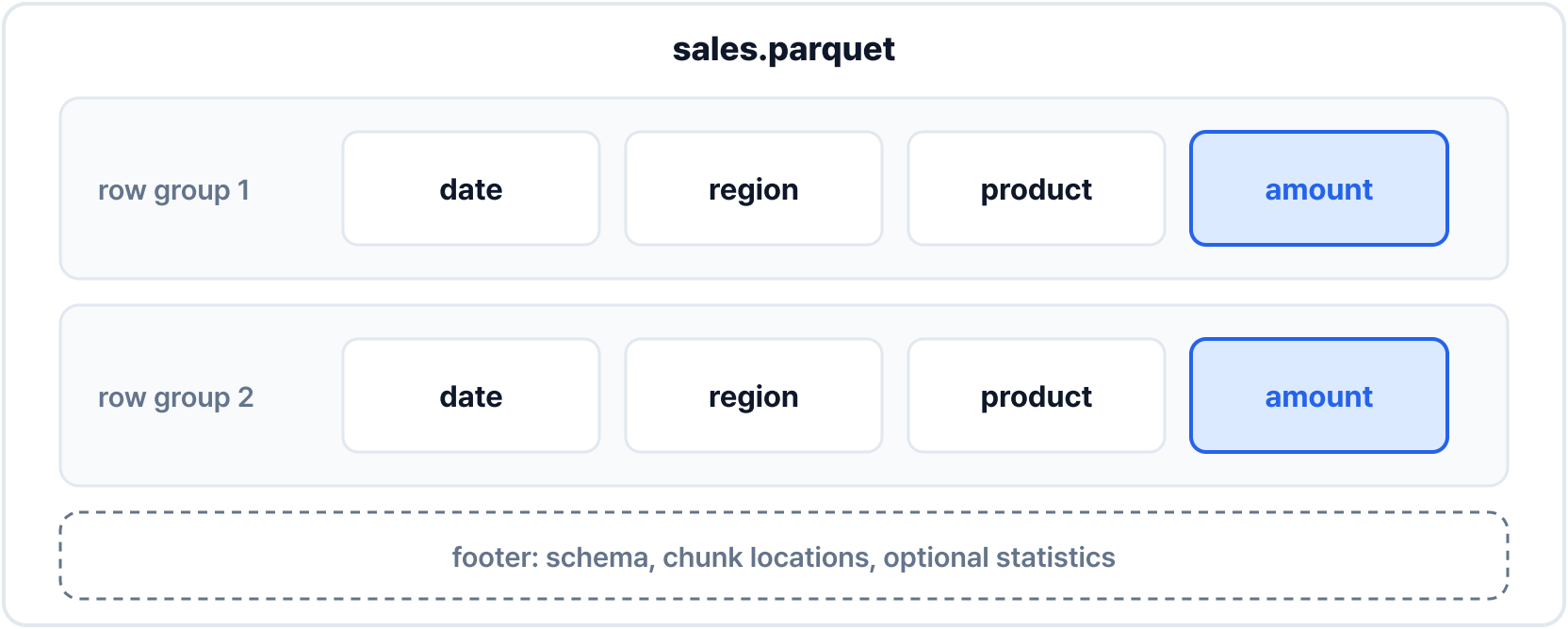
- **Run-length encoding**: (value, count)
  - `A A A A B B C` → `(A,4) (B,2) (C,1)`
- **Dictionary encoding**: distinct strings become small integers
  - `India, Germany, US` → `0, 1, 2` + a dictionary

So the **sort order** sets the file size: long runs come from sorting on the low-cardinality column. Parquet and Capacitor do this per column.

Country (sorted)	RLE
India	(India, 2)
India	
Germany	(Germany, 1)
US	(US, 1)

One type per column and equal values adjacent: **the sort order sets the file size**

Parquet: Columns on Disk



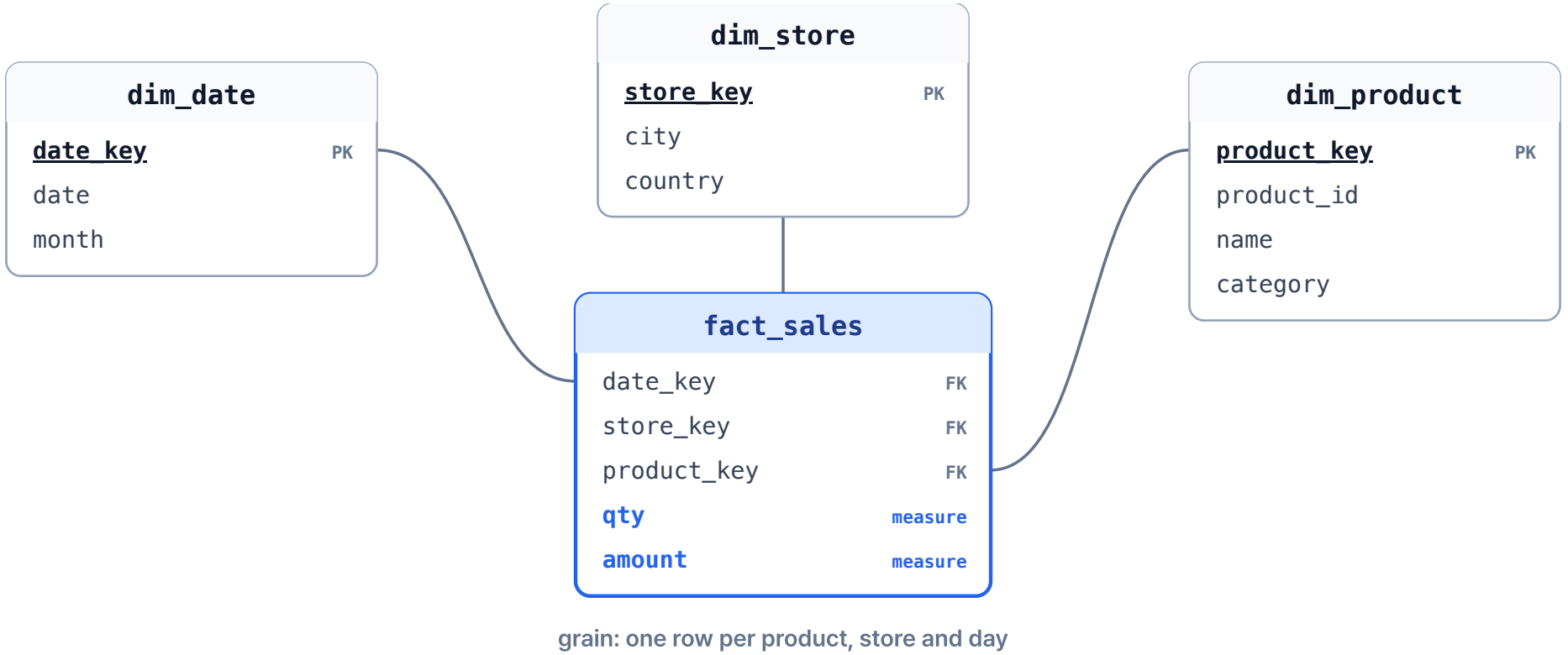
Read projected columns; matching **filter bounds** can skip entire row groups

OLTP and OLAP Compared

Feature	OLTP	OLAP
Full form	Online Transaction Processing	Online Analytical Processing
Purpose	Manages daily transactions in real time	Analyzes large volumes of historical data
Operations	Frequent modifications: INSERT, UPDATE, DELETE	Complex queries: SELECT, aggregations
Storage	Row-based (stores complete records)	Column-based (stores by field / column)
Optimization	Write-optimized	Read-optimized
Response time	Milliseconds	Seconds or minutes
Systems	MySQL, PostgreSQL, Oracle, SQL Server	Snowflake, Redshift, BigQuery, Azure Synapse
Real-world usage	Banking, e-commerce, apps	Reports, dashboards, business insights

Star Schema

A star schema links measures at a declared grain to descriptive dimensions



Measures in the fact table, **descriptions** in the dimension tables

Fact Table Grain

Declare what one fact row stands for before choosing measures and dimensions

fact_sales (selected rows)

date_key	store_key	product_key	qty	amount
20250105	1	1	10	500
20250105	1	2	8	800
20250120	2	1	30	1500

Additive measures **roll up** from finer rows; a coarse aggregate **cannot recover** the detail

Grain of this table: **one product, in one store, on one day**

Slowly Changing Dimensions

When a description changes, overwrite it or keep the old row with its dates

Type 1: Overwrite

product_key	product_id	category
2	P2	Dessert

January sales now **also** report as Dessert

Type 2: Add a Row

product_key	product_id	name	category	valid_from	valid_to
2	P2	Ice-cream	Frozen	2025-01-01	2025-02-01
4	P2	Ice-cream	Dessert	2025-02-01	NULL

Each fact uses the version **valid at its event time**

Type 1 **rewrites history**; Type 2 **keeps it** with a new key and dates

Star Schema Query

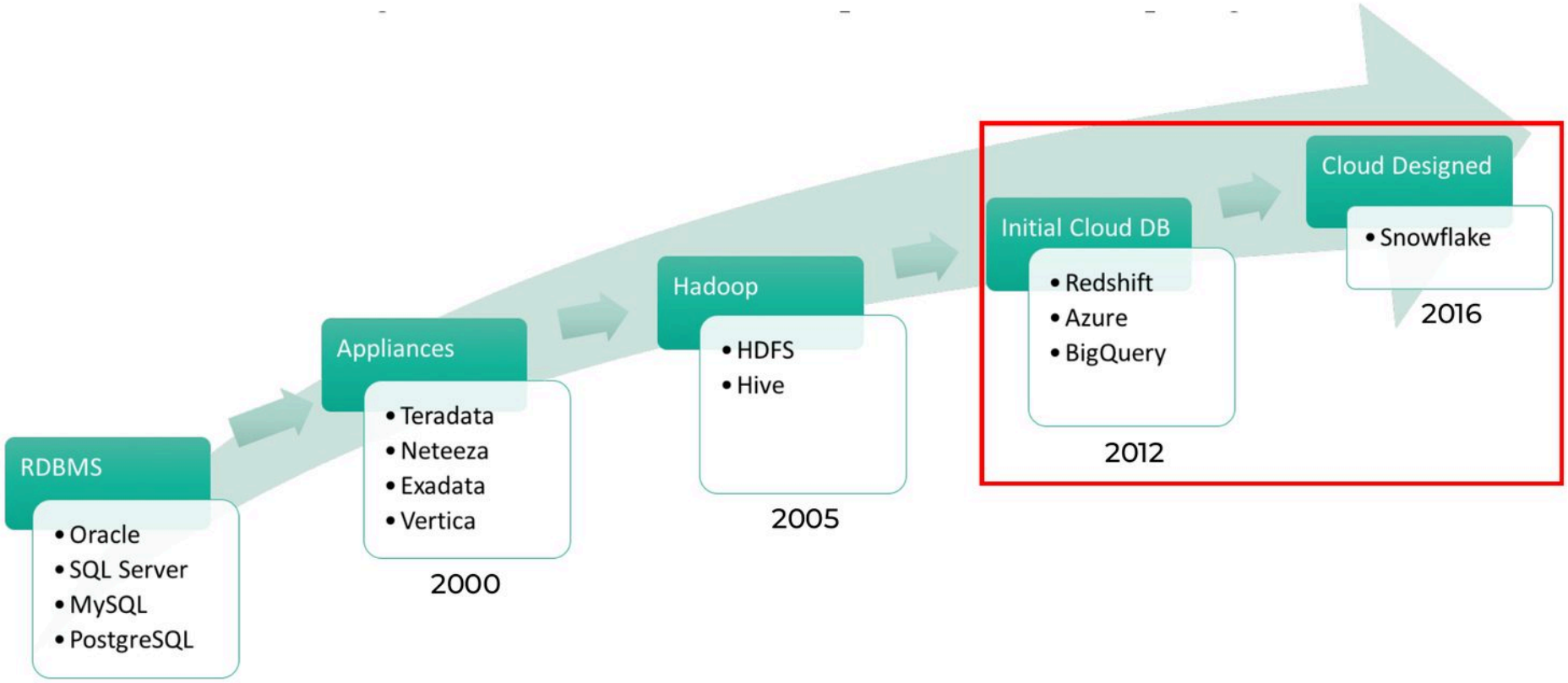
Join the fact table to the dimensions this query needs, then aggregate

```
SELECT   month, category, SUM(amount) AS total
FROM     fact_sales
JOIN     dim_date    USING (date_key)
JOIN     dim_product USING (product_key)
GROUP BY month, category
ORDER BY month, total DESC;
```

month	category	total
2025-01	Snack	2000
2025-01	Frozen	800
2025-01	Meal	300
2025-02	Snack	2500
2025-02	Dessert	1800
2025-02	Meal	200

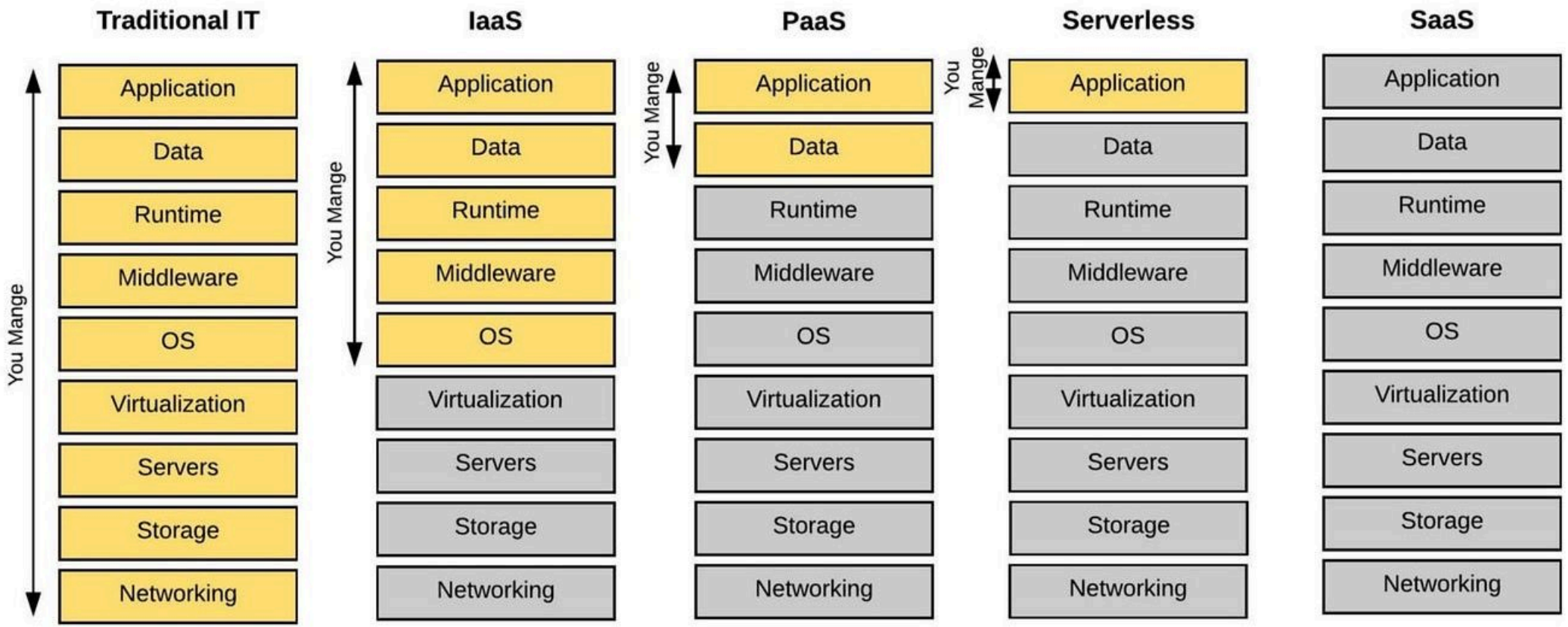
Join the dimensions needed for **grouping or filtering**, then aggregate the **measures**

Managed Infrastructure



From owned hardware to **managed services**; billing and scaling differ by product

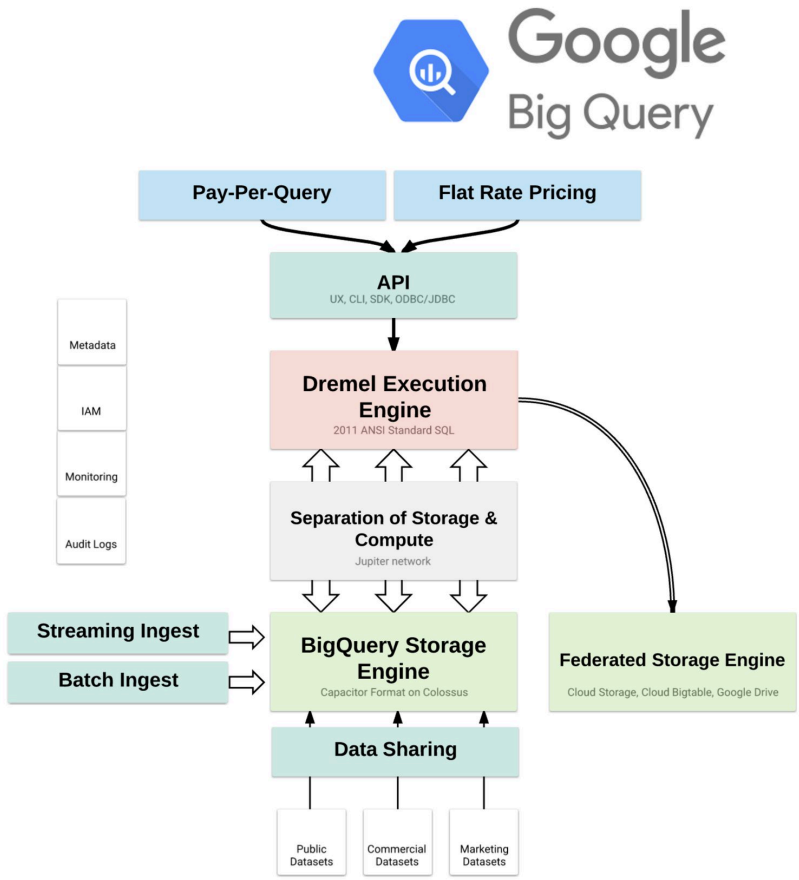
What Serverless Still Leaves You



Serverless removes infrastructure operations; **data, permissions and application behavior** remain your responsibility

BigQuery Storage and Compute

- Fully managed Data Warehouse
  - Large-scale analytics within quotas and resource limits
  - On-demand bytes processed, or capacity-based compute pricing
  - No concept of cluster
- Other similar systems
  - AWS Athena
  - Snowflake
  - ...



Storage and compute are **separate services** joined by a network

BigQuery in the Analytics Flow



Ingest

- Batch & Streaming
- Structured & Semi-structured



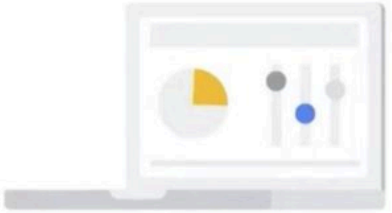
Store

- Up to Petabytes of data
- Datasets & tables
- Federate data



Analyze

- Super-fast SQL queries
- Aggregate
- Transform with DML

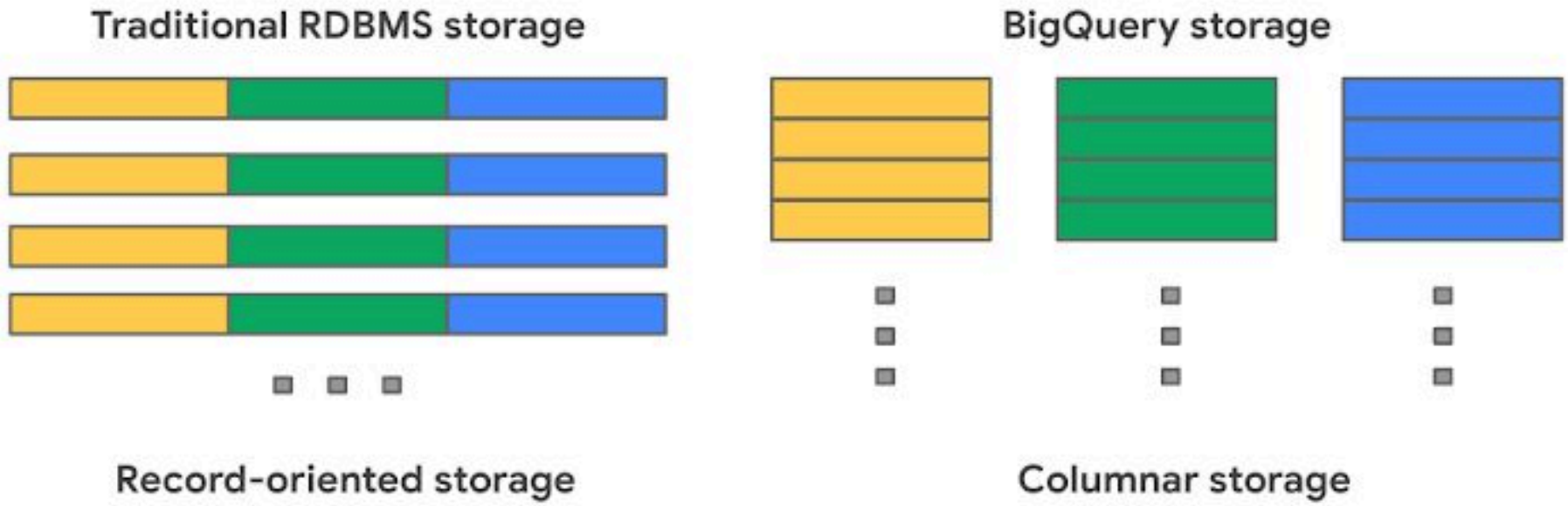


Visualize

- Export and connect to other tools more easily

Ingest, store, analyze, visualize: the whole of Part II in four verbs

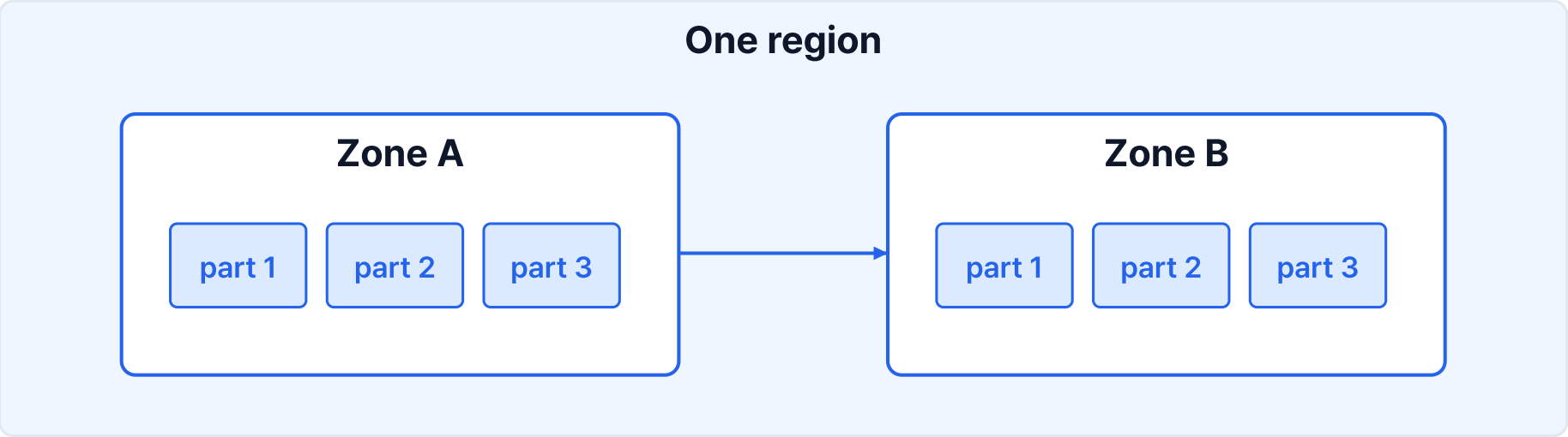
BigQuery = Serverless Data Warehouse



Record-oriented on the left, **columnar** on the right: BigQuery stores the right one

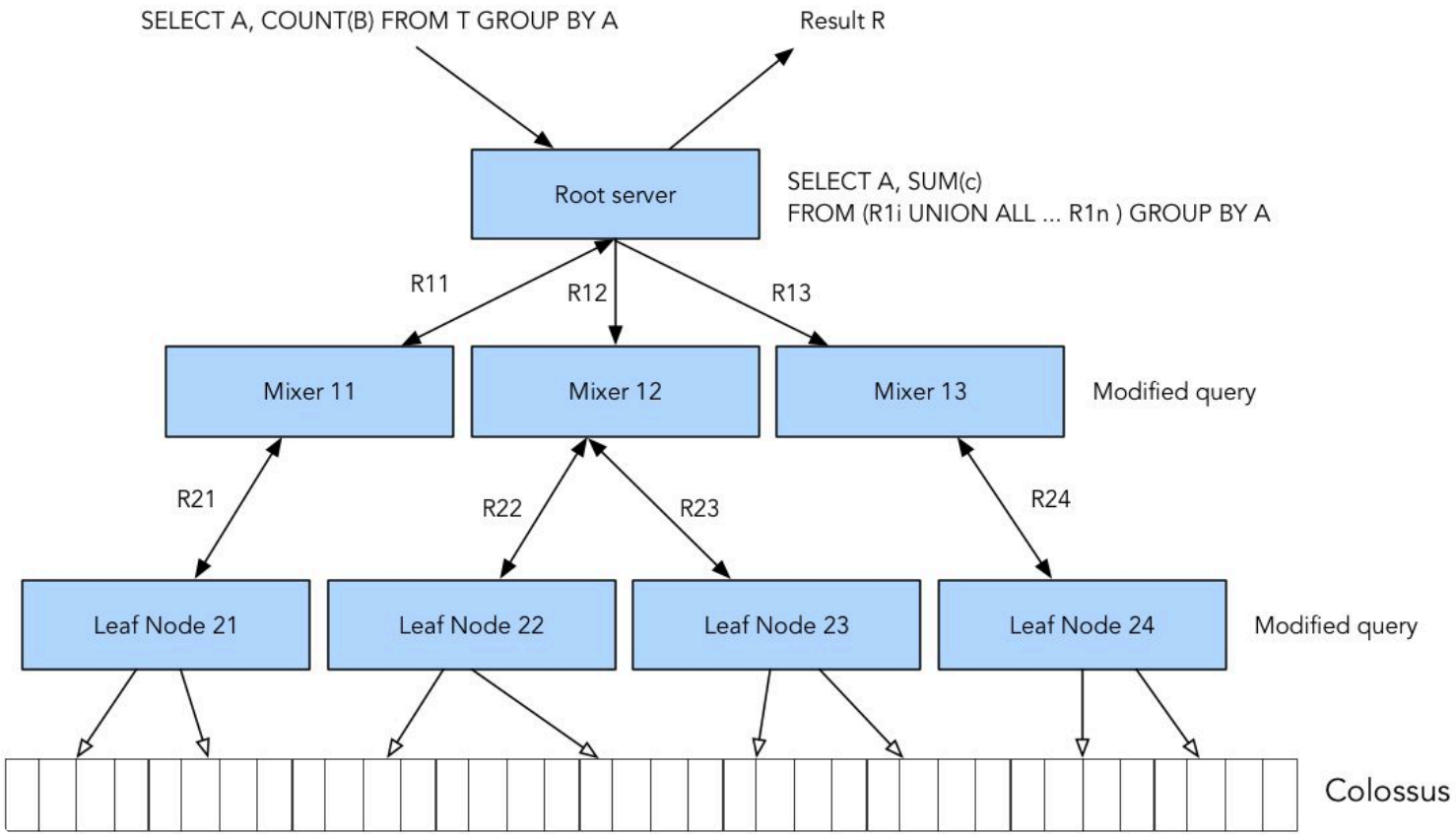
BigQuery Internals (Fault Tolerance)

Synchronous data copies



Data is written to **two zones within one region**; regional disaster recovery is a separate choice

BigQuery Internals (Dremel Engine)



A query flows **down** the serving tree; partial aggregates flow **up**

Programming in BigQuery

How do you program BigQuery?

Programming in BigQuery = SQL

- Basic form (there are many many more bells and whistles)

```
SELECT   <attributes>
FROM     <one or more relations>
WHERE    <conditions>
```

Call this a SFW query.

Demo: OLAP Query

An OLAP query is still SQL: BigQuery on the row-vs-column table

1. Total sales per country, largest first. (The four-row table of the row-store slide.)
2. Which product sold the most across all countries?
3. Which *columns* did each query touch? That is what a column store would read.

```
-- write your query here, then Run
```

QUERY RESULT

Write a query on the left, then Run.

Answer: OLAP Query

```
SELECT   Country,
SUM(Sales) AS total
FROM     Sales
GROUP BY Country
ORDER BY total DESC;
```

Country	total
Germany	4000
India	3000
US	500

```
SELECT   Product,
SUM(Sales) AS total
FROM     Sales
GROUP BY Product
ORDER BY total DESC
LIMIT 1;
```

Product	total
Chocolate	5000

Each query reads **two of the three columns**. This can reduce I/O; the byte ratio depends on column widths, encoding, compression, and pruning.

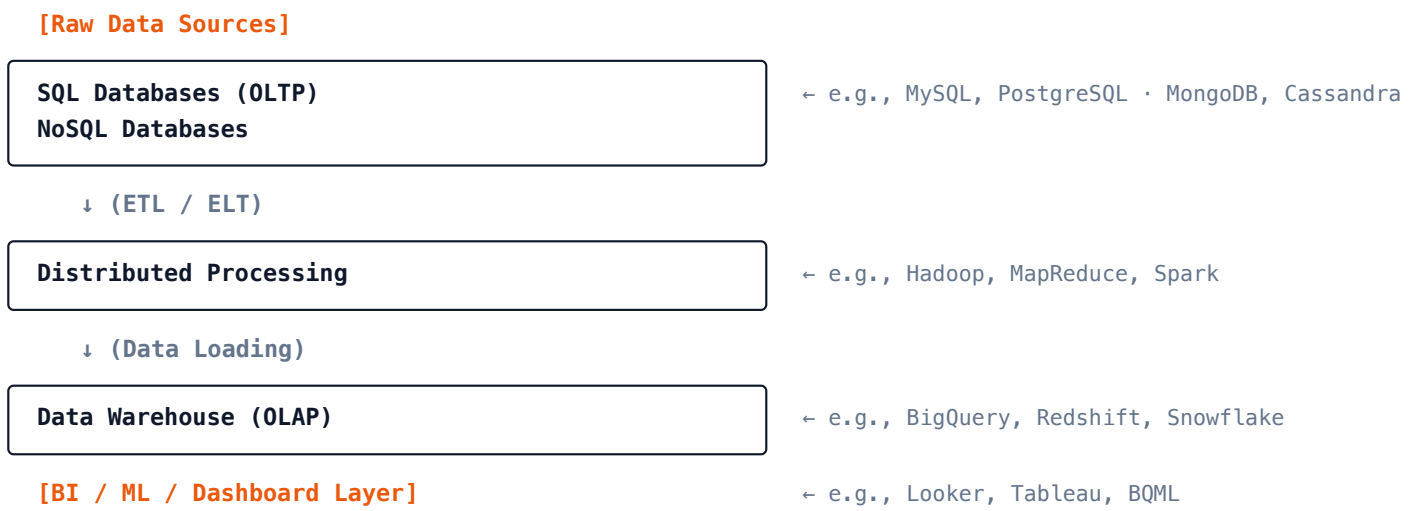
Same SQL as Lecture 8; the **bytes read** are what the warehouse charges for

Choosing a Warehouse

Tool	Runs as	You pay for	Fits when
BigQuery	serverless, Google Cloud	bytes (on-demand) or capacity; storage extra	bursty queries, little ops
Snowflake	virtual warehouses on any big cloud	compute credits and storage	many teams, steady load
Redshift	clusters or serverless, AWS	node/RPU time and storage	data already in AWS
DuckDB	a library on one machine, reads Parquet files directly	no license fee; your compute and storage	data fits on one machine

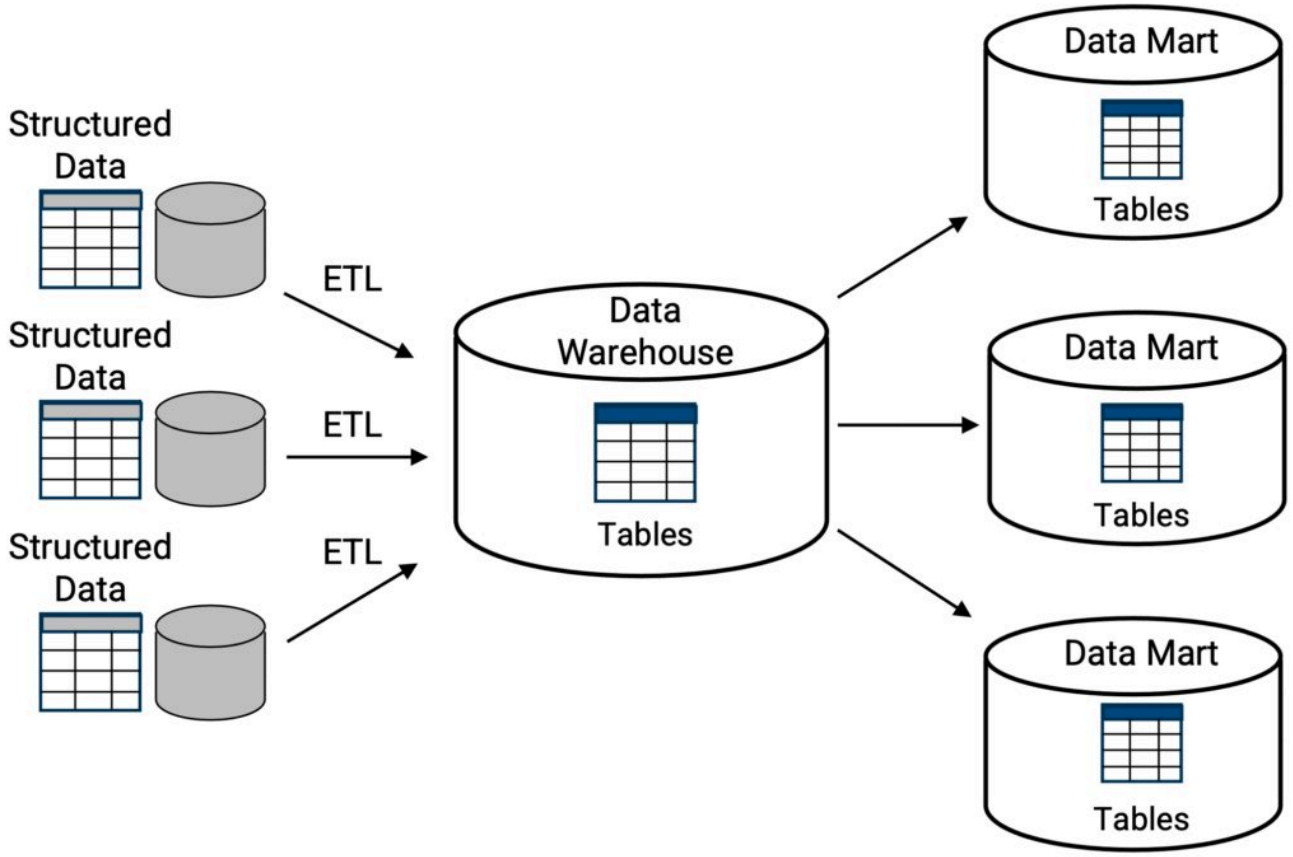
All speak SQL over columns; they differ in **where they run** and **what you pay for**

The Analytics Stack



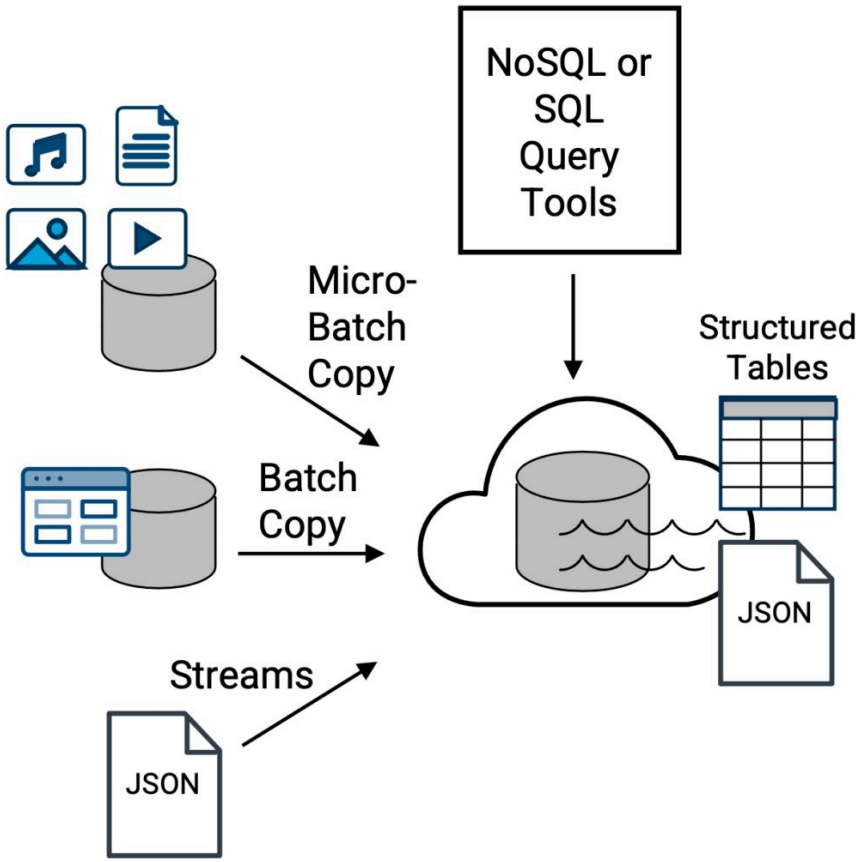
Part II as one stack: sources, OLTP, processing, OLAP, consumers

Recap: Data Warehouse



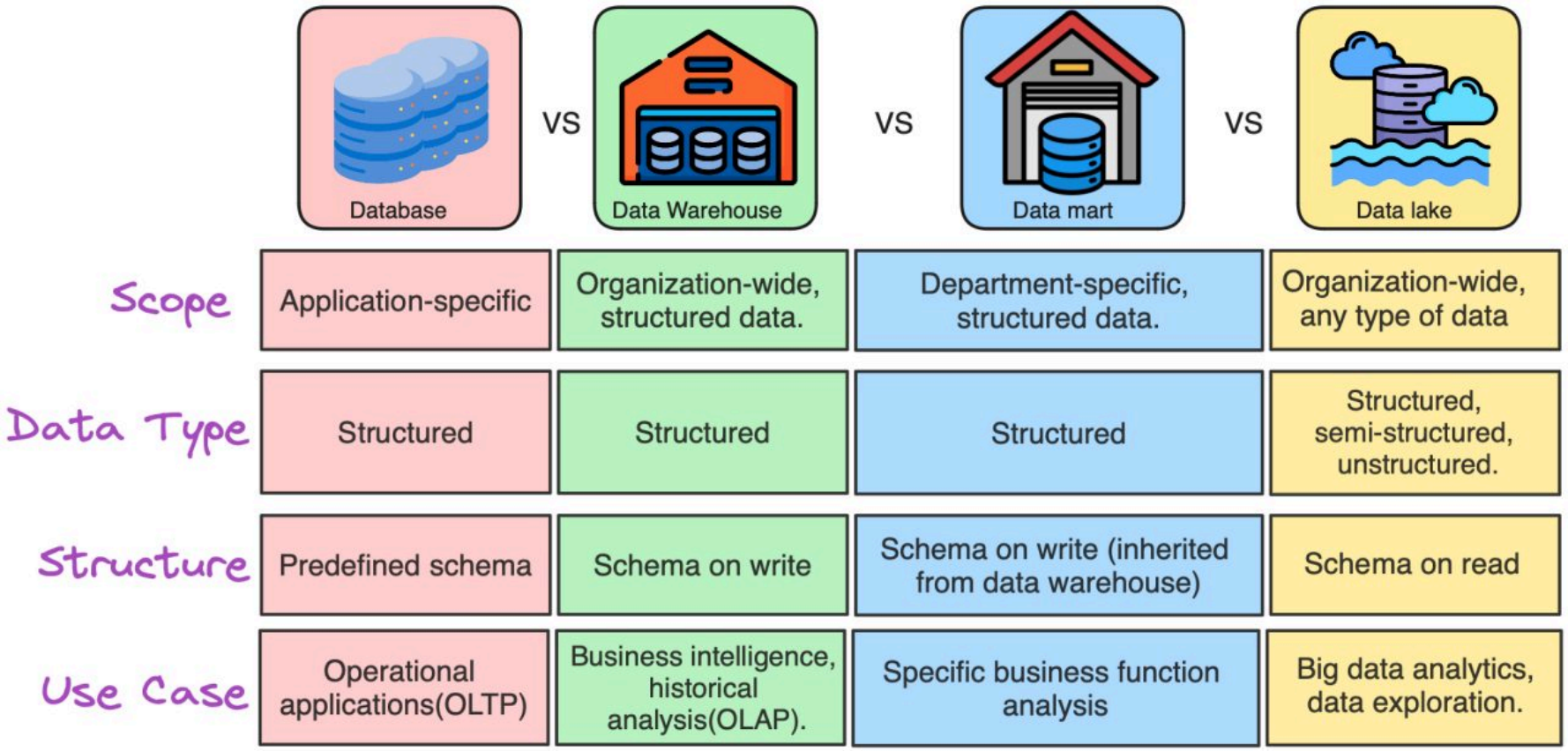
Structured in, structured out: the warehouse **holds tables only**

Data Lake



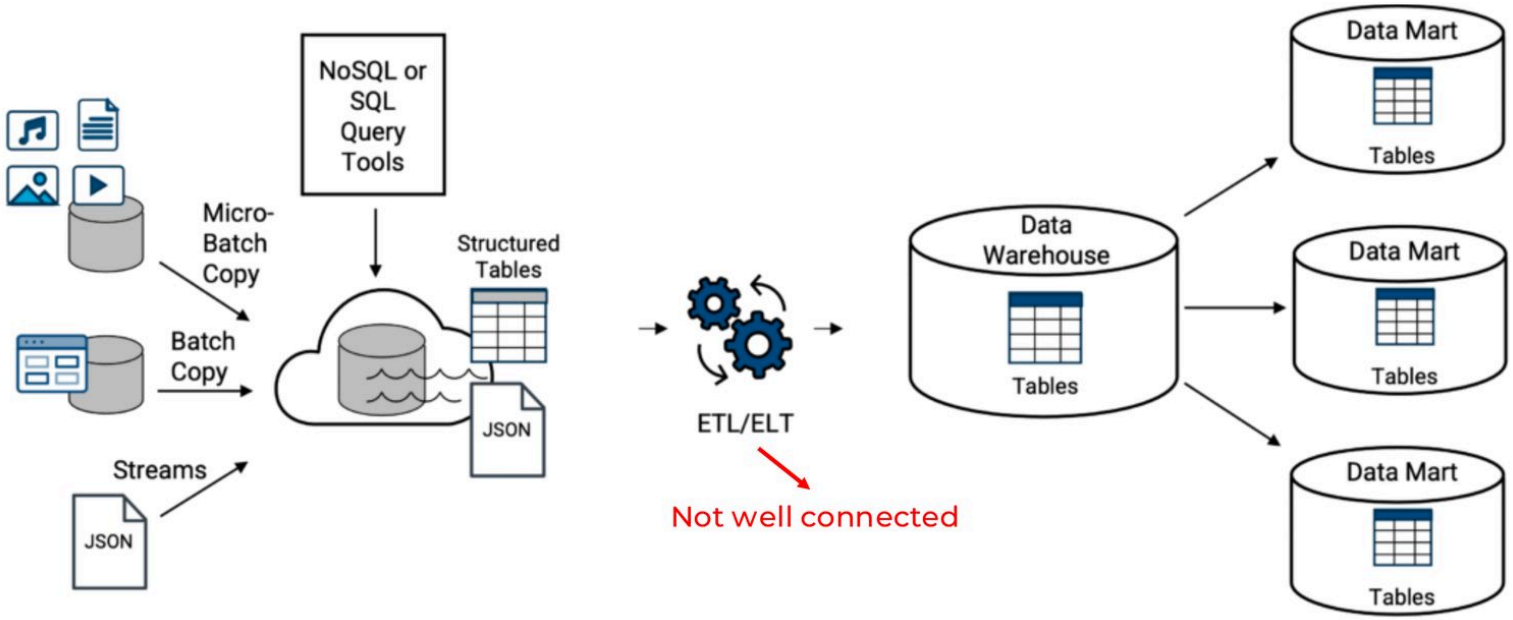
Everything in its native shape, **transformed later**, if at all

Difference Between Data Storages



Scope, type, schema timing, use: **schema on write** vs **schema on read** is the hinge

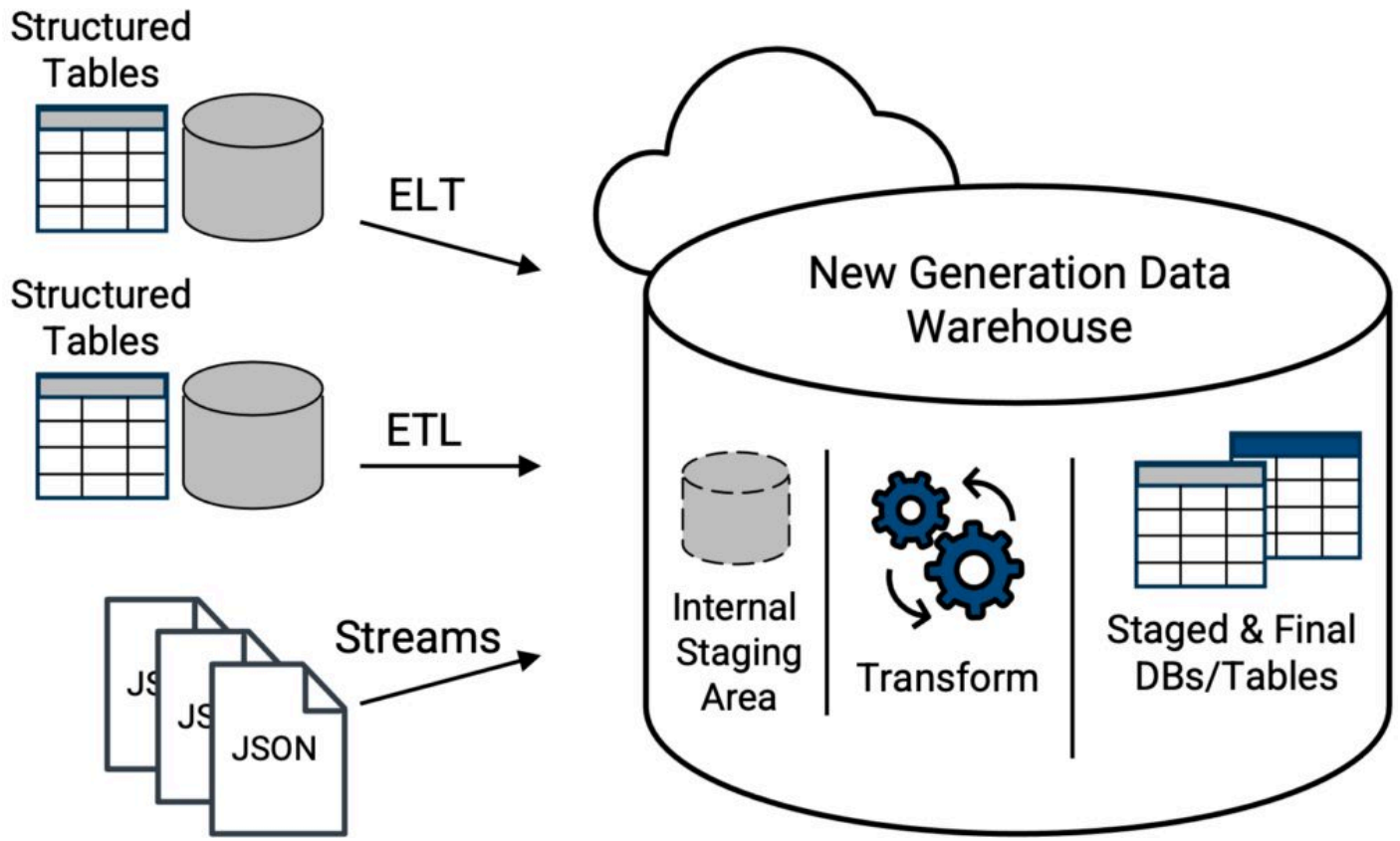
Data Lake → Warehouse



ETL (extract, transform, load): transform *before* the destination, on a separate engine. **ELT** (extract, load, transform): load raw, then transform *inside* the destination, usually as SQL (dbt is the common tool). The difference is **where the T runs**.

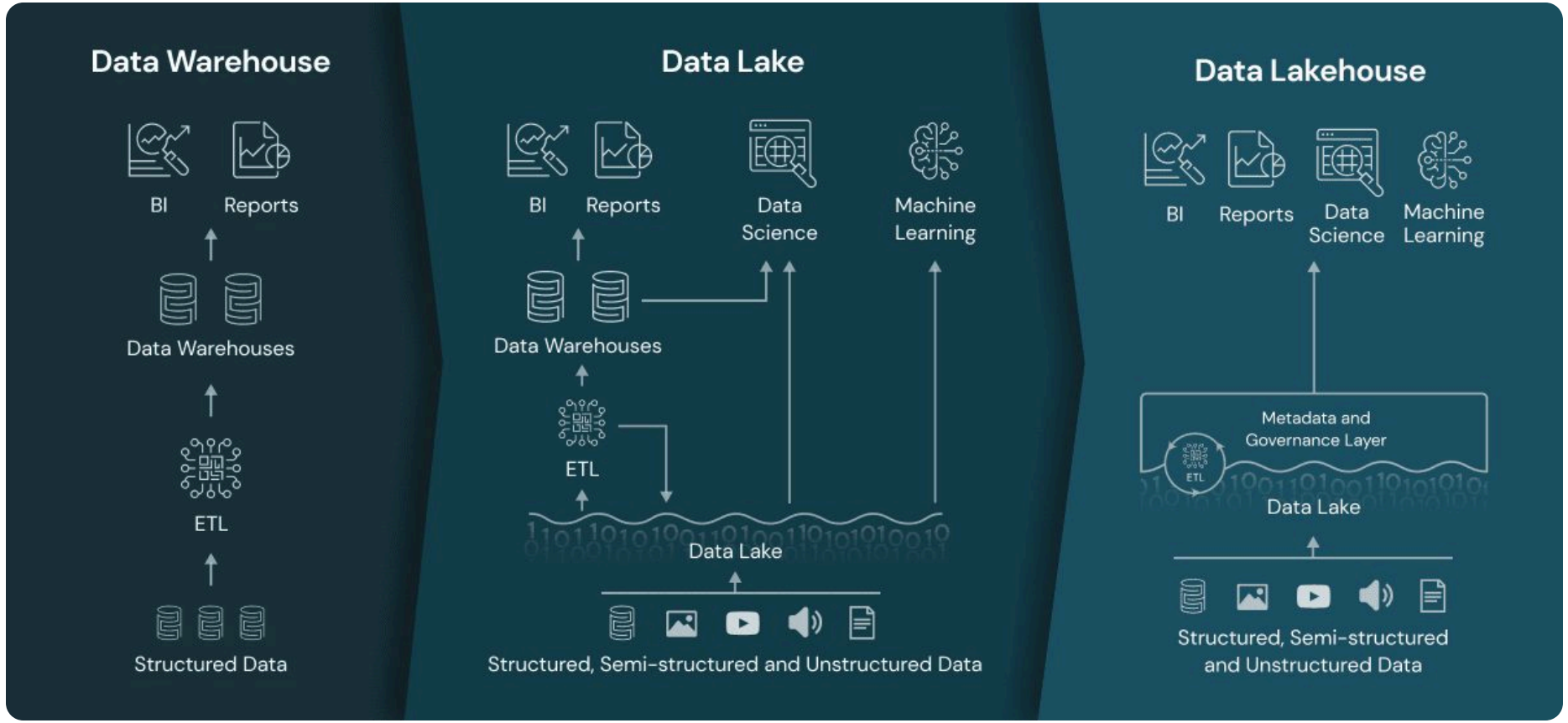
Lake and warehouse stand apart; the T between them runs **outside (ETL) or inside (ELT)** the destination

Lakehouse Storage and Queries



Lake storage underneath, warehouse behaviour on top: **one system**

Lakehouse Metadata Layer



Warehouse, lake, lakehouse: the third puts a **metadata layer** on the lake

Lakehouse Internals

1. **Storage layer:** add transactions, versioning & more
2. **New query engine designs:** great SQL performance on data lake storage systems and file formats
3. **Optimized access for data science & ML**

Open Table Formats

Adds to plain Parquet files	What it means in practice
Transactions	atomic commits; conflicting writers may retry or fail
Schema evolution	add or rename a column without rewriting the files
Time travel	query or restore retained snapshots; retention limits history
Engine interoperability	read/write support depends on format, version and catalog

Iceberg and Delta Lake turn a folder of Parquet files into a **table**

Case study: Netflix's Iceberg Tables

Netflix's legacy Hive tables had no table-wide commit, so Iceberg tracks files in metadata

Legacy Non-Transactional Hive Table

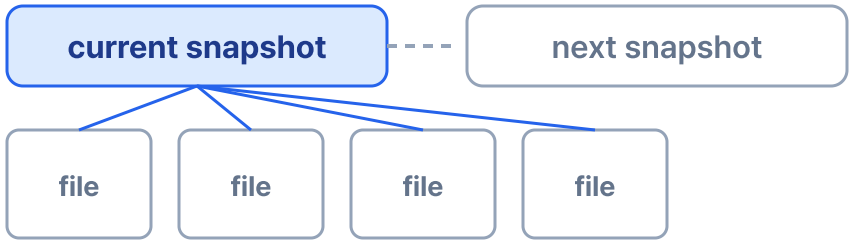


directory-based file discovery

no table-wide snapshot commit

readers can see a write in progress

Iceberg Table = Metadata



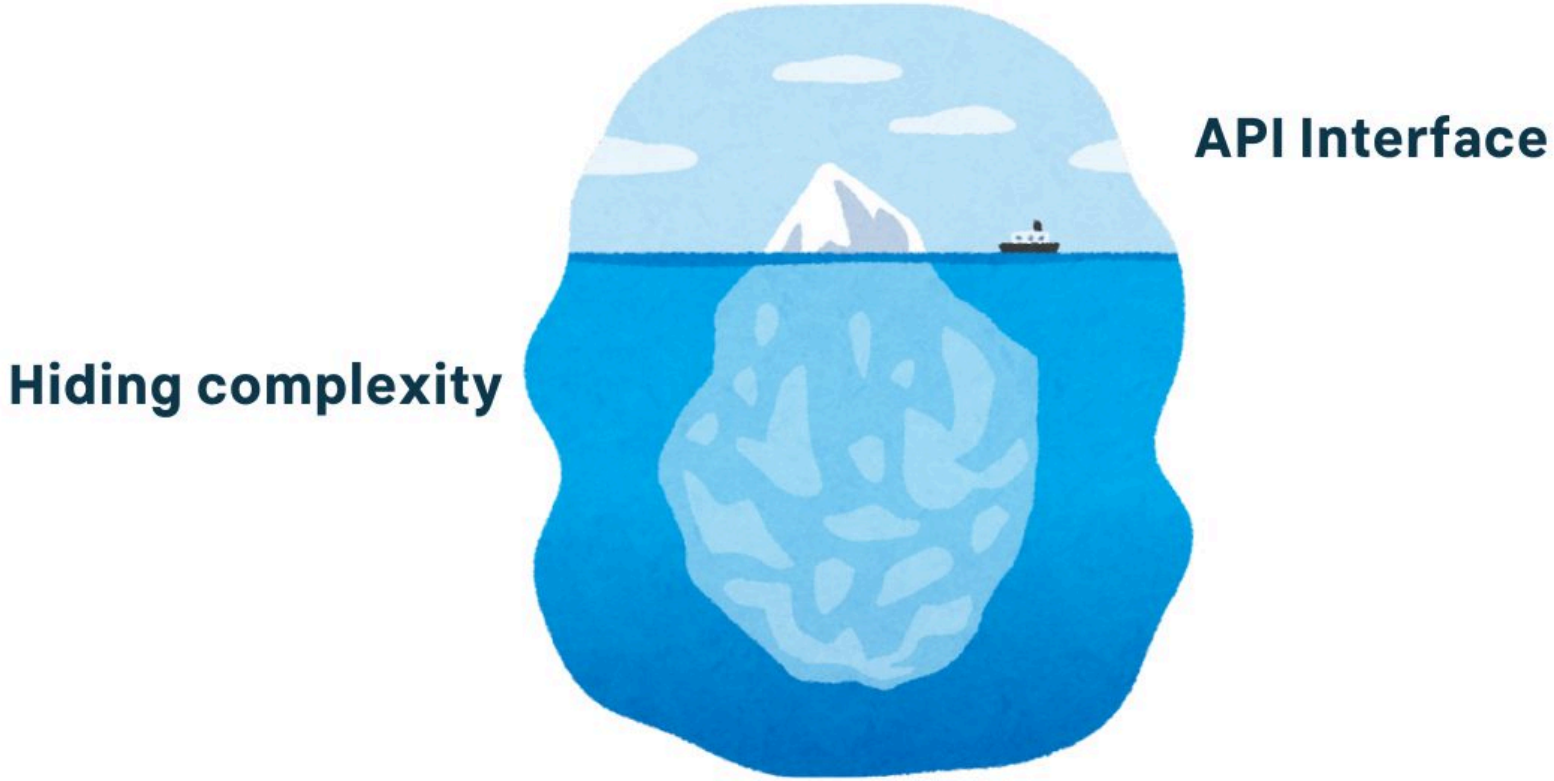
the snapshot lists its files

an atomic metadata commit swaps the pointer

readers see one complete snapshot

An **atomic metadata commit** switches readers from one complete snapshot to another

Abstraction = Hiding Complexity



Above the water an API; below it, everything this lecture described

Data Orchestration

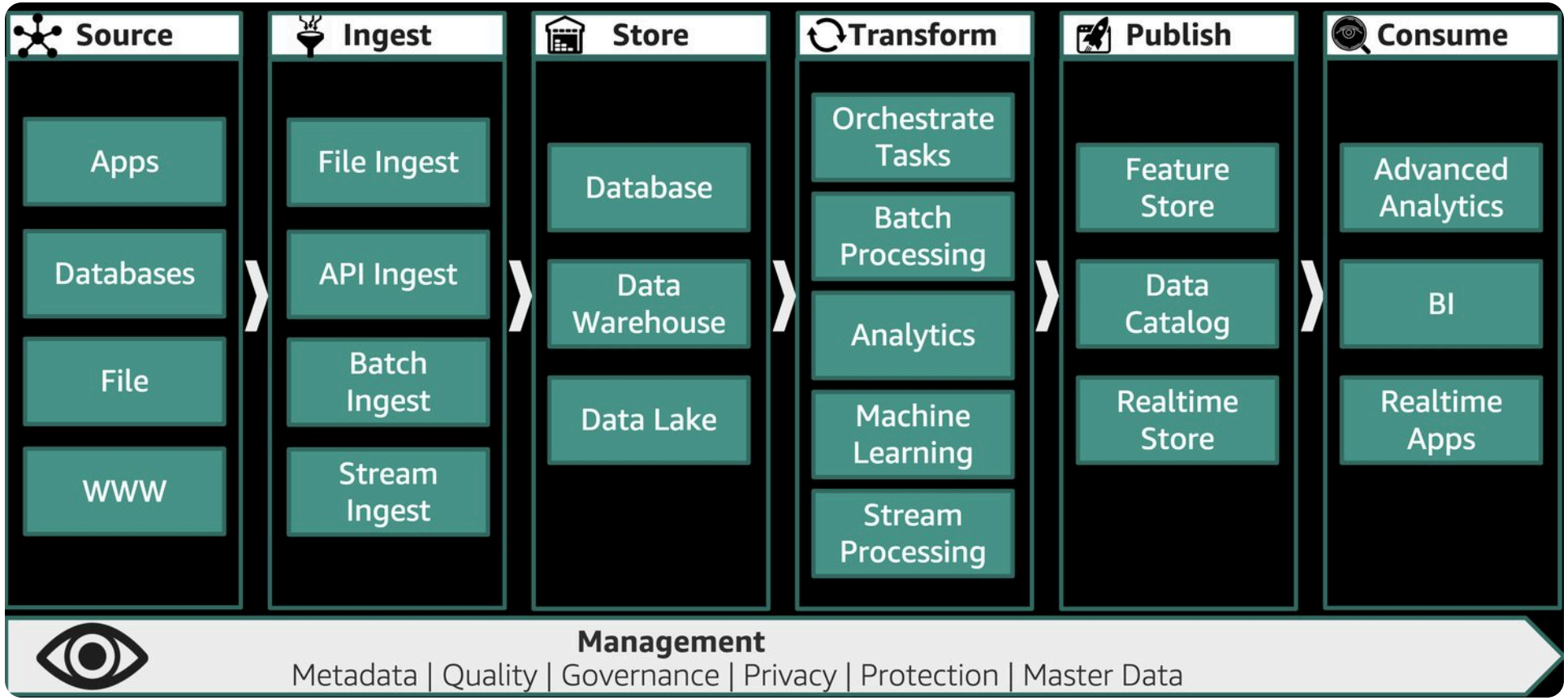
Data **orchestration** is the process of taking siloed data from multiple data storage locations, **combining** and **organizing** it, and making it **available** for data analysis tools. It can also include tasks like **provisioning** resources and monitoring.

Data Pipelines,

Workflows, **J**obs

~~cron~~ a crontab line cannot express “run B after A succeeds, retry twice, alert me”

Data Pipeline



Source, ingest, store, transform, publish, consume: **the course in one row**

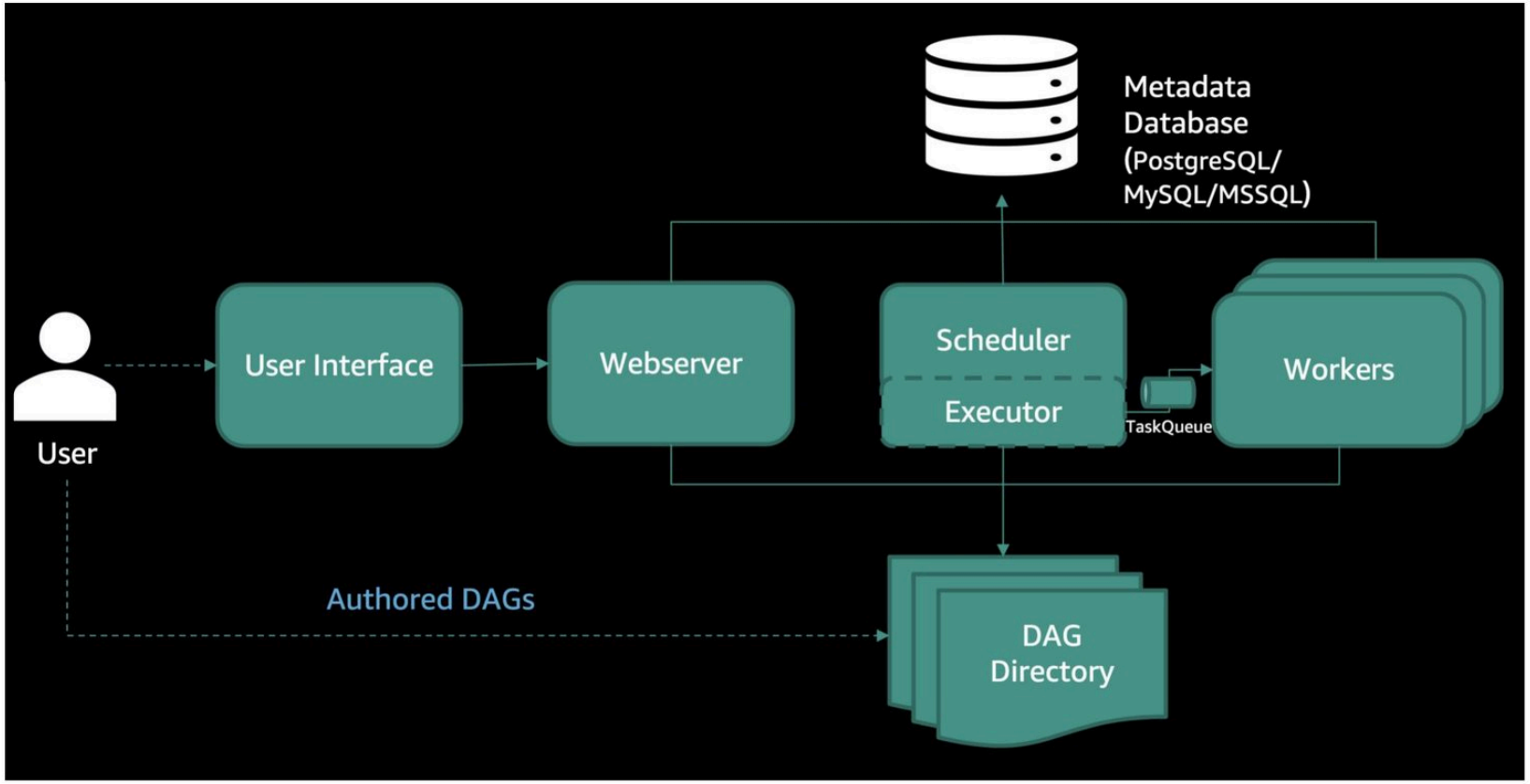
The Tool Should

- Define tasks and their dependencies
  - start, branch, parallel, loop, skip, condition, end
- Trigger them
  - on a schedule, on an event, on a dependency, ad hoc
- Let you act via UI, API, CLI
  - create, start, stop, pause, resume, delete
- Show what happened
  - visualization, metrics, alerts, notification, admin
- Integrate with everything
  - databases, warehouses, lakes, file stores, compute, APIs

+ run anywhere, be available, scalable, reliable, secure, and cheap

That tool is a **workflow orchestrator**. The most used one is Apache Airflow.

Airflow: Architecture



DAG files in, scheduler decides, workers run, **metadata DB remembers**

Airflow DAG

Workflows are called **DAGs** for Directed Acyclic Graph.

Airflow Tasks

- **Tasks:** A task is an instance of an **operator**, or a decorated Python function.
- Tasks can execute work or submit jobs to external systems.
- We tend to classify operators in 3 categories: **Sensors, Operators, Transfers**.

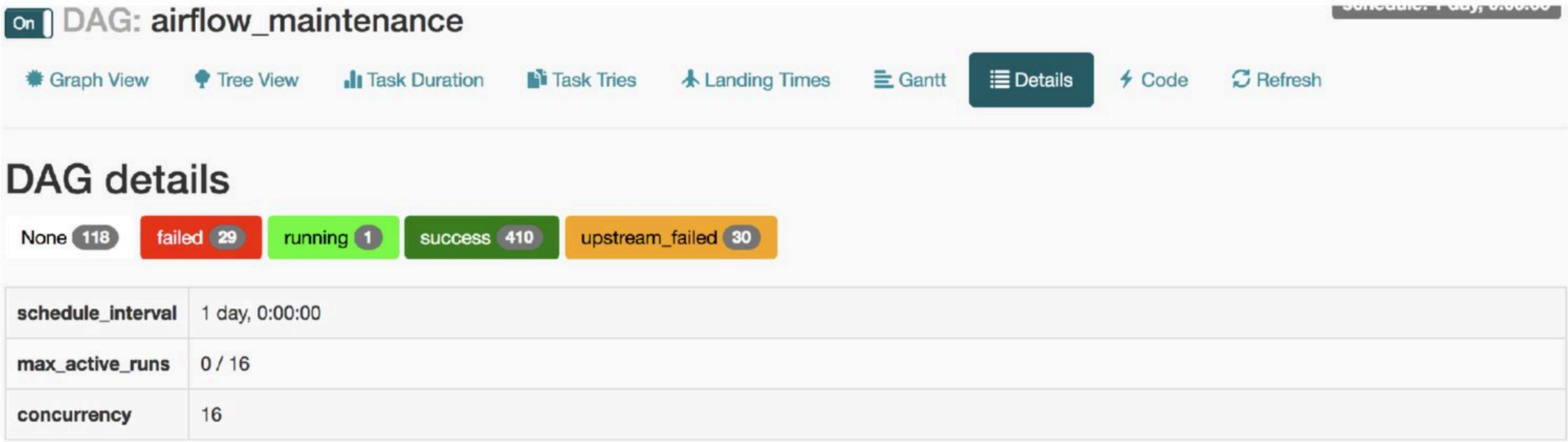
Airflow: Code Example

```
from airflow.sdk import DAG
from airflow.providers.standard.sensors.filesystem import FileSensor
from airflow.providers.standard.operators.empty import EmptyOperator
from datetime import datetime, timedelta, timezone

with DAG("file_sensor_example",
         start_date=datetime(2026, 1, 1, tzinfo=timezone.utc),
         schedule="*/5 * * * *", catchup=False,
         default_args={"retries": 1, "retry_delay": timedelta(minutes=5)}) as dag:
    start = EmptyOperator(task_id="start")
    wait = FileSensor(task_id="wait_for_file", fs_conn_id="fs_default",
                      filepath="input.csv", mode="reschedule",
                      poke_interval=30, timeout=600)
    stop = EmptyOperator(task_id="stop")
    start >> wait >> stop
```

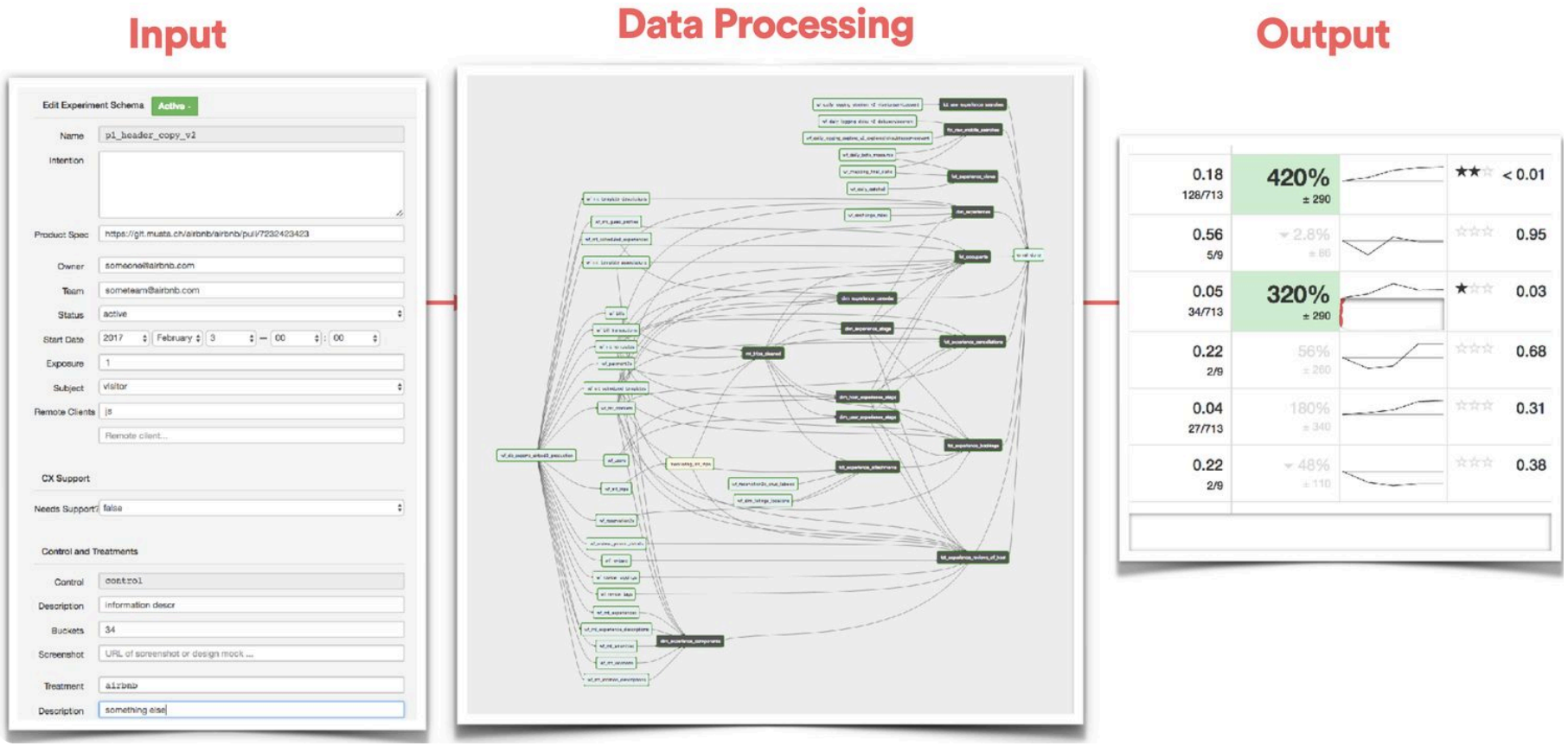
Airflow 3.x + standard provider; configure `fs_default` with the watched folder.

Airflow: Monitoring DAG Status



Historical Airflow UI: run states, schedule and concurrency

Airflow: Common Pattern



Input form, a DAG of processing, output dashboard: **people see only the ends**

dbt: Transformations as SQL Files

```sql
-- models/daily_sales.sql
SELECT order_date, region, SUM(amount) AS total
FROM {{ ref('stg_orders') }}
GROUP BY order_date, region
```

```
$ dbt run     # builds every model, in dependency order
$ dbt build   # builds and tests in DAG order; errors block descendants
```

Each SQL model is one **SELECT**; **ref()** tells dbt the order to build them

Data Quality Checks

```
# models/schema.yml
models:
  - name: daily_sales
    columns:
      - name: order_date
        data_tests: [not_null]
      - name: region
        data_tests:
          - accepted_values:
              arguments:
                values: ['Asia', 'Europe', 'Americas']
```

A pipeline that **finished** is not a pipeline that is **right**; write the checks down

When a Run Fails

Problem	What to do	In Airflow
A task hits a flaky network	try again a few times	retries=3, retry_delay
Yesterday’s run never happened	run it for that date later	backfill, catchup
A rerun would double the rows	make the task idempotent	overwrite the date partition

Plan for failure: **retry**, **backfill**, and make every rerun **idempotent**

Caveat: Non-Idempotent Load

Retrying the append duplicates October 3 and doubles that day's reported sales

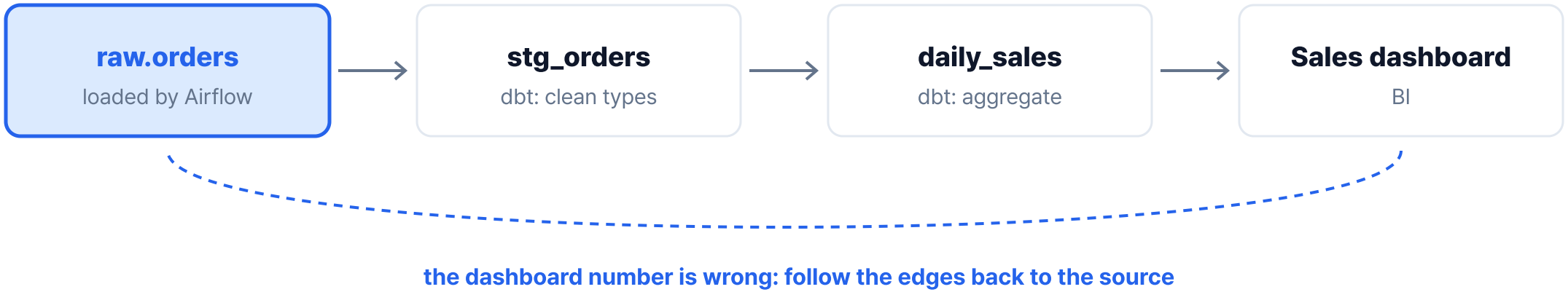
```
INSERT INTO daily_sales            -- the 10-03 run
SELECT order_date, SUM(amount) FROM raw_orders
WHERE order_date = '2025-10-03'
GROUP BY order_date;
INSERT INTO daily_sales            -- retried
SELECT order_date, SUM(amount) FROM raw_orders
WHERE order_date = '2025-10-03'
GROUP BY order_date;
SELECT * FROM daily_sales;
```

order_date	total
2025-10-02	500
2025-10-03	500
2025-10-03	500

Remedy: **atomically replace the date partition**, or **MERGE** one source row per key

Same day, **two rows**: October 3 now reports **double**

Lineage: Which Table Came From Which



Follow the edges **back** to find the cause, **forward** to find what to rebuild

Takeaway: Big Data II

1. Design the **grain and dimension history**; column storage reduces analytical I/O
2. Choose warehouse or lakehouse by **workload and cost**; ELT and open table formats organize stored data
3. **Airflow** schedules the steps and **dbt** builds SQL models; test data, make reruns idempotent, and track lineage